Arquitecturas con superpoderes

Integra agentes de IA en tu arquitectura

sofka_

Orquestaciones Inteligentes

sofka_

LangGraph: Construyendo Flujos... pero con Neuronas

Entendamos primero la base de los flujos cognitivos

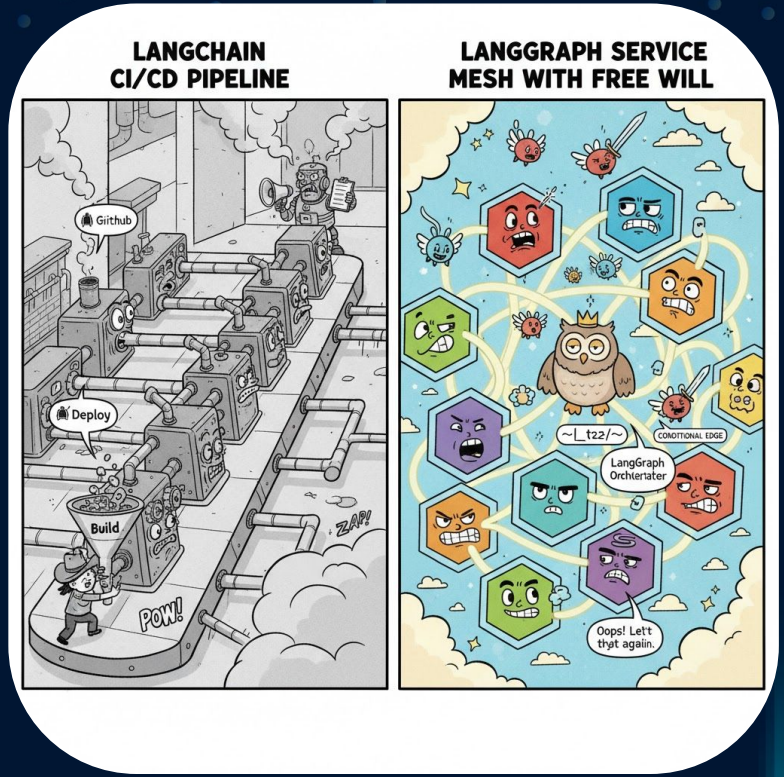
🧠 “Porque los orquestadores merecen una arquitectura bien pensada”

¿Qué es LangGraph?

LangGraph es como el **arquitecto de software** dentro del cerebro de tu IA: organiza flujos, evita cuellos de botella y te ahorra “reuniones de integración”.

- **LangChain** funciona como una **cadena de montaje lineal**: procesa tareas en una secuencia fija, paso a paso, sin la capacidad de desviarse o volver atrás.

- **LangGraph**, en cambio, opera como un **diagrama de flujo inteligente**: cada paso (nodo) puede evaluar la situación y decidir dinámicamente cuál es la siguiente acción a tomar, permitiendo ciclos, bifurcaciones y flujos de trabajo mucho más complejos.



sofka_

Conceptos Fundamentales

1 Estado → “El Data Lake emocional del grafo”

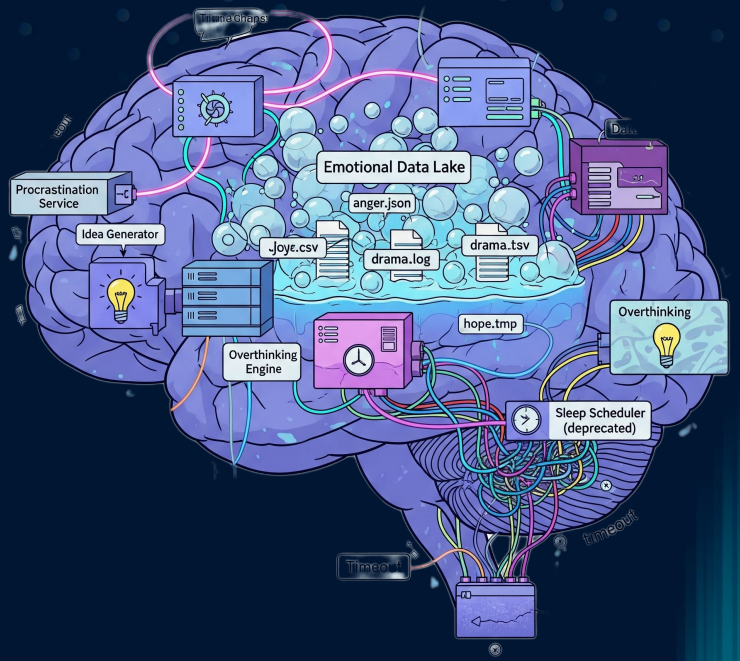
- Contiene lo que el sistema recuerda y arrastra entre llamadas.
- Como un Redis… pero con sentimientos.

2 Nodos → “Los lambda del pensamiento”

- Pequeñas funciones independientes que procesan algo y devuelven valor.
- Cada nodo tiene su propio “bounded context” (y su propia opinión sobre el mundo).

3 Aristas → “Los routers de la red mental”

- Definen hacia dónde fluye la conversación.
- Pueden tener lógica condicional: porque ni los grafos confían ciegamente en los if.



sofka_

Ej: El Sistema de Pedidos ☕



CONTAR ÍTEMS

El becario empieza a sumar... sin saber que esto nunca termina



CONSULTAR AL ASISTENTE CON CAFÉ

El sabio del equipo analiza todo mientras bebe su tercera taza



USAR LA HERRAMIENTA MÁGICA DE INVENTARIO

Un oráculo digital que responde con frases tipo; ⚠ Reabastece pronto" o ✅ Todo bien, joven padawan"



EMITIR EL INFORME FINAL

Un resumen espiritual del caos... con emojis y café incluido



sofka_

FSM Cognitive

sofka_

FSM + LangGraph = “FSM Inteligente”

Un ejemplo práctico para entender

sofka_

¿LangGraph es la herramienta adecuada para orquestar microservicios?

Desafíos con soluciones actuales

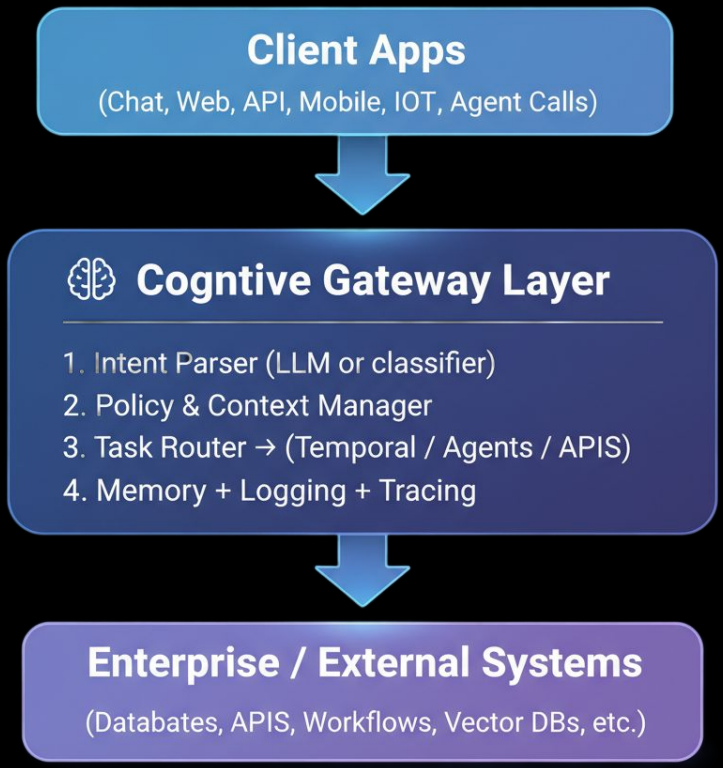
- **Escalabilidad y Resiliencia**
  - Herramientas como LangGraph, Flowise, SuperAGI y n8n no ofrecen **reintentos automáticos**, manejo avanzado de fallos ni escalabilidad nativa a nivel empresarial. LangGraph es particularmente deficiente en mecanismos nativos para esto.
- **Observabilidad y Seguridad**
  - Plataformas como Huginn, Node-RED, Activepieces, LangGraph, Flowise y AutoGen carecen de soporte nativo para **trazas distribuidas, métricas y cumplimiento normativo**, lo que las hace inadecuadas para un gateway empresarial.
- **Producción vs. Prototipos**
  - LangGraph y Flowise son ideales para prototipos, pero no para entornos de producción estables debido a su falta de resiliencia y observabilidad. Make y n8n son más robustos en producción, pero no están optimizados para flujos **multi-agente con IA.**

Capa de Gateway Cognitivo

Pensada y diseñada para **sistemas distribuidos**

sofka_

Un **Cognitive Gateway Layer** es una capa inteligente de orquestación que se sitúa entre tus sistemas empresariales (APIs, bases de datos, microservicios) y los modelos de IA (LLMs, agentes, RAG, etc.).



El Cognitive Gateway Layer convierte peticiones simples (humanas o del sistema) en acciones inteligentes, seguras y auditables.

Convierte tus órdenes en inteligencia... y tus errores en evidencia controlada.

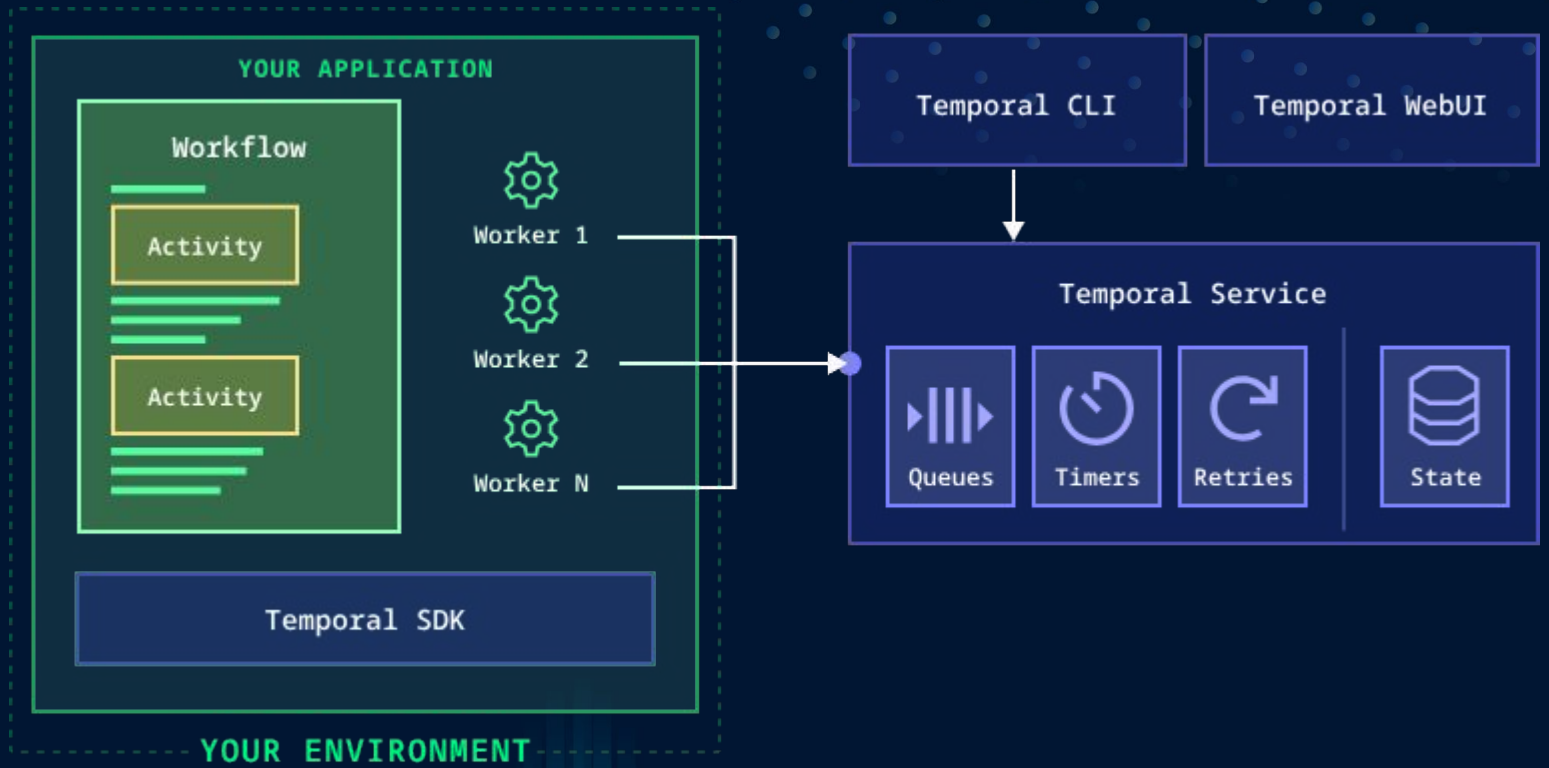
sofka_

¿Qué es Temporal.io?

Temporalio es como un director de orquesta para tus procesos distribuidos 🎻. Cada microservicio toca su parte (tareas, flujos, eventos), y Temporalio se **asegura de que todos entren a tiempo, repitan si fallan y mantengan el ritmo,** sin que tú tengas que preocuparte de los errores o los tiempos muertos.



Temporal


YOUR APPLICATION
Workflow
Activity
Activity
Worker 1
Worker 2
Worker N
Temporal SDK
YOUR ENVIRONMENT
Temporal CLI
Temporal WebUI
Temporal Service
Queues
Timers
Retries
State

Principio clave	Cómo lo cumple Temporalio	Metáfora
Resiliencia	Reintenta tareas automáticamente y mantiene el estado incluso tras fallos.	Si un músico se equivoca, el director lo hace repetir sin parar el concierto. 🎼
Durabilidad y recuperación	Los workflows son persistentes y pueden continuar tras reinicios o despliegues.	Si se va la luz, la orquesta sigue donde quedó. 💡
Desacoplamiento	Cada workflow y activity vive separado, facilitando independencia y despliegues autónomos.	Cada músico ensaya su parte sin depender de los otros. 🎻
Escalabilidad	Permite distribuir y balancear las activities en múltiples workers.	Más músicos, mismo ritmo. 🎺
Orquestación confiable	Coordina flujos complejos de manera determinista, auditable y asíncrona.	El director mantiene la armonía y el ritmo de la sinfonía. 🕴️🎵

Cognitive Gateway con LLM + Temporal + Embeddings

No hay nada como un buen ejemplo para entender

sofka_

Act. Retrieve context from transactions

Crea índice y recupera contexto

sofka_

Act. Generate response

Llama al LLM para generar la respuesta final.

RAG con Vector Store (FAISS)

- **Vectorización**: Conversión de transacciones a textos embeddings.
  - Embeddings convierten búsqueda léxica (keywords) en búsqueda semántica (significado)
- **Índice FAISS**: Almacenamiento vectorial para búsqueda semántica eficiente.
  - FAISS optimiza búsqueda en millones de vectores
- **Retriever**: Recupera top-K (k=2) documentos más relevantes a la query.
- **Contexto**: Unión de documentos recuperados para generación de respuestas.

Flujo: Textos ➜ Embeddings ➜ FAISS Index ➜ Búsqueda Semántica ➜ Contexto RAG

Patrón: Retrieval-Augmented Generation (RAG) con búsqueda vectorial local.

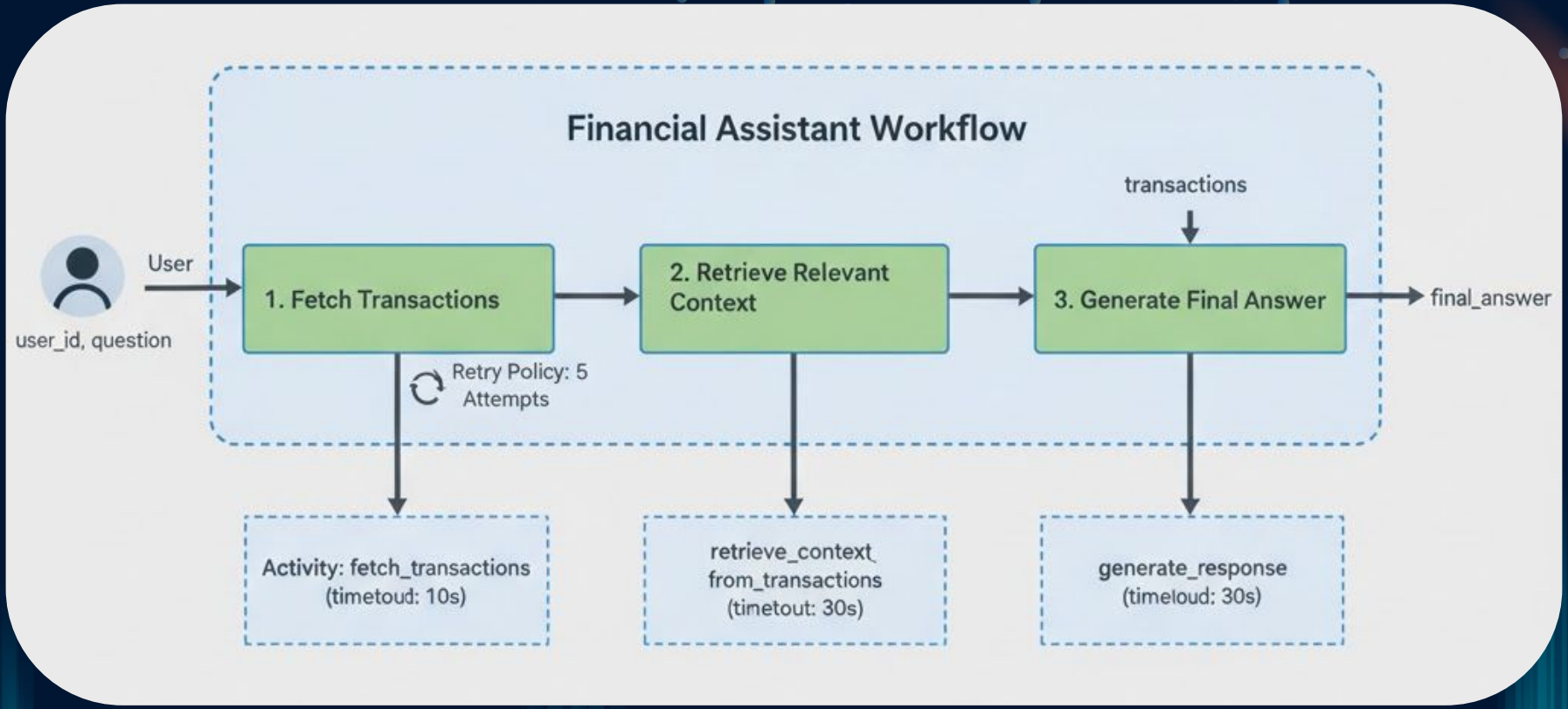
Pipeline de Inferencia LLM

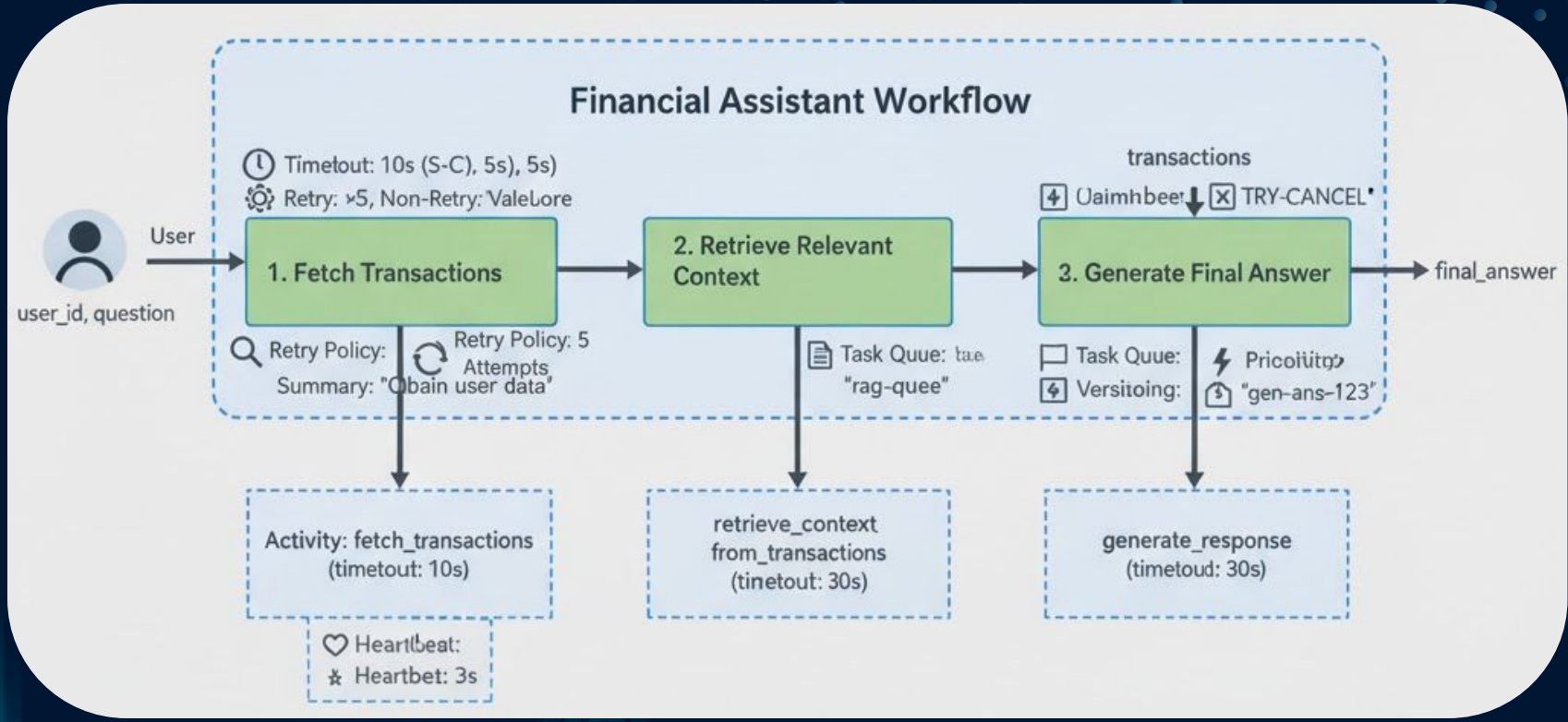
- ❏ **Prompt Engineering**: ChatPromptTemplate con rol de sistema (asesor financiero) + template dinámico para inyectar {question} y {context} (RAG).
- ❏ **Cadena de Procesamiento**: prompt | llm | StrOutputParser() - Composición modular de componentes LangChain.
- ❏ **Ejecución Asíncrona**: @activity.defn para workflows distribuidos, invocación via chain.invoke().
- ❏ **Entrada/Salida**: Recibe contexto vectorial + consulta ➜ Devuelve respuesta estructurada.

Flujo: RAG ➜ Template ➜ LLM ➜ Parsing ➜ Respuesta textual

Output: "El gasto en supermercado fue de 75.50 € el 12 de octubre."

Arquitectura desacoplada, escalable y orquestable.


Financial Assistant Workflow
transactions
User
user_id, question
1. Fetch Transactions
2. Retrieve Relevant Context
3. Generate Final Answer
final_answer
Retry Policy: 5 Attempts
Activity: fetch_transactions (timetoud: 10s)
retrieve_context_from_transactions (timetout: 30s)
generate_response (timeloud: 30s)


Financial Assistant Workflow
Timetout: 10s (S-C), 5s), 5s)
Retry: ×5, Non-Retry: ValeLore
transactions
TRY-CANCEL
User
user_id, question
1. Fetch Transactions
2. Retrieve Relevant Context
3. Generate Final Answer
final_answer
Retry Policy:
Retry Policy: 5 Attempts
Summary: "Obain user data"
Task Quue:
"rag-quee"
Task Quue:
Versitoing:
"gen-ans-123"
Activity: fetch_transactions
(timetout: 10s)
retrieve_context
from_transactions
(tinetout: 30s)
generate_response
(timetoud: 30s)
Heartbeat:
Heartbet: 3s

sofka_

Resiliencia y Fiabilidad

Gestión de Ejecución y Tiempos en Temporal:

- **Definición de Colas:** Asigna actividades a colas para escalabilidad y organización de workers.
- **Límite de Tiempo Total (End-to-End):** Tiempo máximo para completar una actividad desde su programación hasta su fin, evitando estancamientos.
- **Tiempo de Recolección (Schedule-to-Start Timeout)**: Plazo máximo para que un worker inicie una actividad programada, identificando escasez de workers o colas congestionadas.
- **Tiempo de Ejecución (Start-to-Close Timeout)**: Limita la duración de la lógica de negocio de una actividad iniciada.
- **Heartbeats para Actividades Largas:** Señales periódicas del worker para detectar inactividad y permitir reanudación.

Manejo de Errores y Reintentos:

- **Configuración de Reintentos:** Estrategia de reintentos de Temporal para actividades fallidas, esencial para la resiliencia ante fallos transitorios (red, servicios externos). Incluye intervalos, coeficientes de backoff y número máximo de intentos.

Comportamiento y Contexto

Comportamiento y Despliegue

- Tipo de cancelación: Define cómo se comporta la actividad al cancelar su flujo de trabajo (ej. TRY_CANCEL, WAIT_CANCELLATION_COMPLETED, ABANDON), lo que permite controlar la limpieza de recursos.
- Intención de versionado: Guía a Temporal sobre cómo manejar las versiones de la actividad durante el despliegue, asegurando que se utilice la versión correcta en los flujos de trabajo.

Identificación y Observabilidad

- ID de actividad: Un identificador único opcional para instancias de actividad, útil para la depuración y auditoría.
- Resumen: Una breve descripción de la actividad que mejora la observabilidad en la interfaz de usuario y los registros de Temporal.
- Prioridad: Asigna prioridad a la actividad en la cola de tareas, permitiendo que las tareas críticas se procesen primero.

¿Por qué Temporal para tu arquitectura?

- Agentes, MCP y Pipelines de IA
- Humanos en el ciclo (Humans-in-the-Loop)
- Patrones Compensatorios (Saga)
- Workflows de larga duración
- Registros duraderos (Durable Ledgers)
- CI/CD confiable
- Adquisición de clientes
- Cumplimiento de pedidos (Order Fulfillment)

Gracias
Raul. A. Alzate

Solutions Architect



sofka_